

BayesianFramework Cheat Sheet

Purpose: BayesianFramework provides a compact interface for hierarchical Bayesian modeling with NumPyro, including global parameters, optional correlated entry-level local parameters, MCMC training, held-out prediction, test-time local inference, scoring, reproducible RNG control, and persistence.

Dependencies: jax, numpyro, numpy, cloudpickle

Installation: Install in editable mode with `pip install -e "/path/to/BayesianFramework"` then import with `from BayesianFramework import BayesianFramework`. Editable installation means updates pulled into the repository are available without reinstalling. If using VS Code and the import is not recognized by autocomplete, add `"python.analysis.extraPaths": ["/path/to/BayesianFramework"]` to settings.json.

Conventions

`arg{}` - dictionary whose values have no entry dimension; used for one entry's data, local parameters, or model output, and for global parameters or other unbatched named values.

`arg{[]}` - dictionary containing arrays spanning multiple entries; returned model outputs may also include a leading posterior/sample dimension.

`arg[]` - 1D sequence.

`Fn(...)` -> value - function argument or method together with the value it returns.

`A | B` - either input/return type A or input/return type B, depending on behavior.

Italic arguments - accept None as a valid input.

Main Functions

BayesianFramework(modelDataFull{[]}, ModelFn(globalParams{[]}, localParams{[]}, data{[]}) -> modelOut{[]}, localParamNames{[]}, GlobalParamFn(dataShapes{[]}) -> globalParams{[]}, choleskyConcentration, rngSeed) -> BayesianFramework: Creates a framework object. **modelDataFull** contains the full dataset. **ModelFn** evaluates one entry. **localParamNames** optionally defines standardized entry-level parameters; when multiple locals are present, their correlation is learned with an LKJ prior controlled by **choleskyConcentration**, a positive number. **GlobalParamFn** optionally receives a dictionary of full-data array shapes and defines model-facing global parameters. **rngSeed** is an optional nonnegative integer that initializes the persistent RNG state used by calls that do not provide their own seed. **Returns:** a new BayesianFramework object.

Train(trainIndices{[]}, TrainLikelihoodFn(globalParams{[]}, localParams{[]}, data{[]}, modelOut{[]}), numWarmup, numSamples, num_chains, acceptProb, dense_mass, medianSamples, printSummary, rngSeed) -> MCMC: Fits the entries selected by **trainIndices** using NUTS MCMC. **TrainLikelihoodFn** defines the likelihood from the globals, locals, selected data, and model output. **numWarmup**, **numSamples**, **num_chains**, and **medianSamples** are positive integers; **acceptProb** is a probability; **dense_mass** and **printSummary** are booleans. **rngSeed** is an optional nonnegative integer that overrides the persistent RNG for this call without advancing the persistent RNG state. **Returns:** the NumPyro MCMC object.

Test(testIndices{[]}, TestLikelihoodFn(globalParams{[]}, localParams{[]}, data{[]}, modelOut{[]}), nPosteriorSamples, numWarmup, numSamples, num_chains, acceptProb, dense_mass, medianSamples, nTrajectorySamples, printSummary, rngSeed) -> modelOut{[]} | MCMC{[]}: Evaluates the trained model on entries selected by **testIndices. If **TestLikelihoodFn** is omitted, no test MCMC is run: globals are drawn from the training posterior, new locals are sampled from their population distribution, and **nTrajectorySamples** trajectories are generated. If **TestLikelihoodFn** is provided, test MCMC infers the test-entry locals conditional on globals from the training posterior; **nPosteriorSamples** is an optional positive integer controlling how many training-posterior global states are tested. **numWarmup**, **numSamples**, **num_chains**, and **medianSamples** are positive integers; **acceptProb** is a probability; **dense_mass** and **printSummary** are booleans. **nTrajectorySamples** is an optional positive integer controlling the number of generated trajectories. **rngSeed** is an optional nonnegative integer that overrides the persistent RNG for this call without advancing the persistent RNG state. **Returns:** sampled modelOut{[]}] without MCMC, or a list of NumPyro MCMC objects with MCMC.**

ScoreTrain(ScoreFn(modelOut{[]}, data{[]}) -> score{[]}) -> scores{[]}: Scores stored training predictions. ScoreFn evaluates one entry and may return a single value, an array, or a dictionary of values/arrays. ScoreTrain() applies it to every posterior sample and training entry and collects the results.

ScoreTest(ScoreFn(modelOut{[]}, data{[]}) -> score{[]}) -> scores{[]}: Scores the most recent test predictions. ScoreFn evaluates one entry and may return a single value, an array, or a dictionary of values/arrays. ScoreTest() applies it to every sample and test entry and collects the results.

GetTrainModelOutput() -> modelOut{[]}: Retrieves stored training model outputs. Returns: training model outputs with posterior-sample and entry dimensions.

GetTestModelOutput() -> modelOut{[]}: Retrieves stored outputs from the most recent test. Returns: test model outputs with sample and entry dimensions.

SetRng(rngSeed) -> None: Sets or resets the persistent RNG state using **rngSeed**, a nonnegative integer. Subsequent **Train()** and **Test()** calls use and advance this state unless they receive their own **rngSeed**; a call-specific seed overrides but does not advance the persistent state.

GetRecord(print) -> record{[]}: Collects framework metadata; print is a boolean controlling whether it is pretty-printed. Returns: a dictionary containing parameter names, train/test indices, MCMC settings, RNG state, and function names.

Save(fileName) -> None: Serializes the complete framework object, including trained state and functions, to **fileName**, a file-path string, using `cloudpickle`. Load only trusted saved files.

BayesianFramework.Load(fileName) -> BayesianFramework: Loads an object previously written by **Save()**; **fileName** is a file-path string. **Returns:** the loaded BayesianFramework object.